

LINGUAGEM JAVA - DESENVOLVENDO COM JAVA II

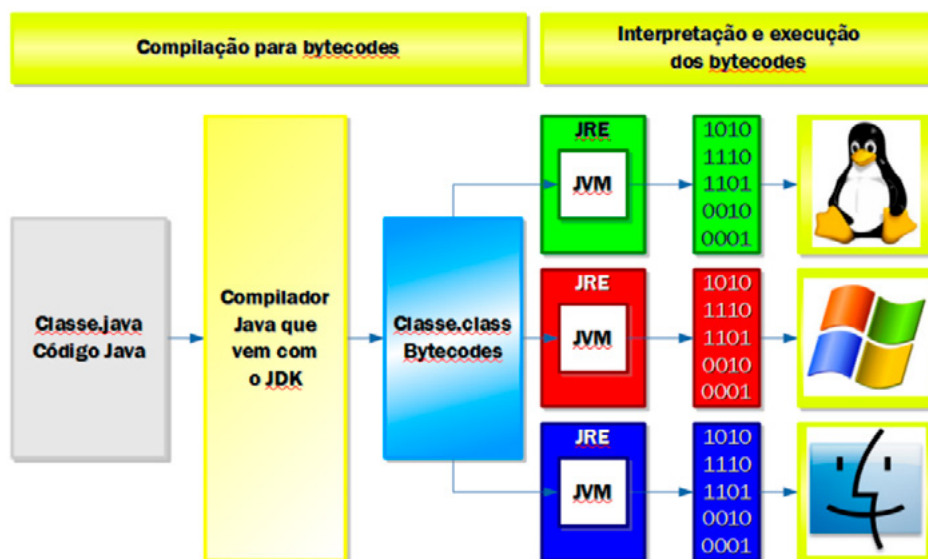
Neste bloco, o foco será em abordar o desenvolvimento de aplicações em Java, detalhando o processo que vai da criação à execução. Inicialmente, discute-se o processo de compilação, que requer a recompilação do programa para cada plataforma específica em que se deseja rodá-lo. Embora essa prática melhore o desempenho ao otimizar a aplicação para uma plataforma, resulta em perda de portabilidade.

Por exemplo, na ação de recompilar um programa para diferentes sistemas operacionais, como Windows, Linux e Unix, a fim de disponibilizar a aplicação executável para todos os usuários, tem-se um processo comparado a um multiverso, em que cada plataforma possui seus próprios “heróis”, representados pelo compilador e pelo editor de ligação.

Além disso, discute-se a interpretação, destacando que o interpretador, diferente do compilador, não produz um executável, mas atua como uma instância de software que lê, interpreta e executa cada linha do programa. Assim, em aplicações interpretadas, cada instrução é lida e executada em tempo real para que a aplicação funcione adequadamente.

No Java, ocorre um processo híbrido que combina tanto a compilação quanto a interpretação. Essa característica é fundamental para entender a dinâmica de execução das aplicações desenvolvidas na linguagem. A seguir, serão apresentados os detalhes que justificam essa classificação do Java como híbrido.

COMPILAÇÃO, INTERPRETAÇÃO E EXECUÇÃO



Um programador deve baixar um JDK, que é um kit de desenvolvimento Java, o qual inclui o compilador Java e a própria linguagem de programação. Após desenvolver sua aplicação e escrever o código, que consiste em classes com a extensão .java, é importante notar que todos esses elementos podem ser questões de concursos.

Quando se cria uma classe, o nome dessa classe define o nome do arquivo. Por exemplo, a classe "Professor" é armazenada no arquivo "Professor.java". Assim, "Aluno" será "Aluno.java", "Pessoa" será "Pessoa.java" e "ContaCorrente" será "ContaCorrente.java".

Ao compilar essas classes com o comando "javac", arquivos com a extensão.class são gerados. Por exemplo, ao compilar "Professor.java" com `javac Professor.java`, um arquivo "Professor.class" será criado, contendo bytecodes, que é uma linguagem intermediária do Java. Esses bytecodes não são executáveis diretamente em uma plataforma específica; eles passam por um processo adicional. A primeira fase do processo Java envolve a compilação do código fonte (.java) para bytecodes (.class) pelo compilador Java C incluído no JDK.

Para executar uma aplicação.class em uma plataforma específica, é necessário baixar um JRE (Java Runtime Environment) para essa plataforma. Por exemplo, se a plataforma for Linux, deve-se instalar o JRE correspondente, que inclui uma JVM (Java Virtual Machine) e uma API para auxiliar na execução. A JVM é responsável por ler, interpretar e executar os bytecodes contidos nos arquivos.class.

Assim, a aplicação será executada sobre a plataforma Java, que integra a JVM e a API. Para Windows, é necessário baixar o JRE específico que inclui a JVM correspondente. Da mesma forma, para macOS, deve-se instalar o JRE adequado para essa plataforma.

Os bytecodes representam um conjunto que será interpretado e executado em cada plataforma que tenha um JRE. Com isso, não é necessário recompilar o código para cada plataforma; basta que ela tenha um JRE instalado. Essa necessidade de escolha do JRE adequado é evidenciada quando se acessa o site da Oracle, onde se deve selecionar entre as opções para Windows, Linux ou macOS.

Suponha que um indivíduo propõe reescrever sua aplicação em Java, abandonando a linguagem compilada. Na fase inicial da execução, ele compila os arquivos.java, como professor.java, questão.java e banca.java, utilizando o comando javac para gerar os respectivos arquivos.class. Cada arquivo.class contém bytecodes, e esse processo de compilação é realizado uma única vez.

O autor instrui os alunos a baixar o JRE específico de suas máquinas no site da Oracle, informando que a aplicação funcionará sobre esse JRE, que inclui a JVM, responsável por ler, interpretar e executar a aplicação. Caso um aluno, após as férias, solicite a aplicação para sua máquina Unix, o autor não precisa recompilar. Ele simplesmente orienta o aluno a baixar o JRE adequado e a acessar um link para baixar a aplicação.

Assim, não há necessidade de recompilar o código, pois a aplicação já foi compilada para uma linguagem intermediária. Cada aluno poderá executar a aplicação em sua máquina, desde que tenha o JRE apropriado. Essa abordagem reflete o slogan da Sun Microsystems, adquirida pela Oracle, que enfatiza a portabilidade da linguagem Java. Essa questão gera um amplo leque de tópicos que podem ser abordados em exames.



5m



10m

Em síntese, é possível elaborar um `arquivo.java` em Java, em uma linguagem de alto nível, utilizando código que é legível e editável. O compilador Java, representado pelo comando `javac`, gera um `arquivo.class` a partir do `arquivo.java`. Por exemplo, ao criar a classe `professor`, é gerado automaticamente o `arquivo professor.java`. Ao compilar com o comando `javac`, obtém-se o `arquivo professor.class`, que contém os `bytecodes`.

Esses `bytecodes` representam uma linguagem intermediária, própria da JVM. Para acessar a JVM, é necessário ter o JRE específico da plataforma. O JRE é responsável por ler, interpretar e executar os `arquivos.class` com `bytecodes` na plataforma correspondente. Essa estrutura forma uma camada de abstração, permitindo que a aplicação Java funcione sobre a plataforma específica. Em resumo, esse é o funcionamento básico do processo.

ARQUIVOS.JAVA E.CLASS

Os `arquivos.java` são arquivos de texto que podem ser abertos até mesmo em editores simples, como o Notepad, e contêm código fonte. Em contrapartida, os `arquivos.class` são gerados a partir da compilação dos `arquivos.java` pelo compilador `javac`. Por exemplo, uma classe `professor` estará contida em um arquivo chamado `professor.java`. Após a compilação, será gerado o `arquivo professor.class`, que contém os `bytecodes` dessa classe.



15m

ARQUIVOS.CLASS

Os `arquivos.class` não contêm código nativo para processadores específicos, mas sim `bytecodes`, que são uma linguagem intermediária reconhecida pela JVM. O comando `javac` é utilizado para compilar `arquivos.java`, enquanto o comando `java` serve para chamar a JVM e executar `arquivos.class`.

A extensão `.class` não é necessária ao executar o arquivo; basta utilizar o comando `java` seguido do nome do arquivo, por exemplo, `"java professor"`. Dessa forma, para interpretar e executar a aplicação contida nos `arquivos.class`, invoca-se o comando `java`, que cria uma instância da JVM e permite a execução da aplicação Java.

JDK (JAVA DEVELOPMENT KIT)

Um programador deve baixar o JDK, que é o Kit de Desenvolvimento Java, um conjunto de ferramentas essenciais para o desenvolvimento de aplicações Java. O JDK inclui uma JRE, que é o ambiente de execução, permitindo que o programador teste suas aplicações. Além disso, o JDK contém uma JVM e uma API para auxiliar na execução das aplicações.

Essa plataforma Java é fundamental para testar os aplicativos desenvolvidos. O JDK também oferece outras ferramentas, como o Java C para compilação, o Java para execução (que invoca a JVM), um Applet Viewer para visualizar applets, um Javadoc para gerar

documentação e um JAR para empacotar aplicações. Assim, o JDK proporciona diversas ferramentas de programação e um ambiente de execução Java.

JRE (JAVA RUNTIME ENVIRONMENT)

O JRE deve ser baixado apenas por clientes, enquanto programadores devem optar pelo JDK, que já inclui o JRE correspondente à sua plataforma. Por exemplo, ao utilizar Linux, deve-se baixar o JDK para Linux, que contém o JRE e a JVM específicos dessa plataforma.

O JRE é o ambiente de execução Java, essencial para a operação das aplicações Java. Para executar uma aplicação distribuída, o usuário deve baixar o JRE, que representa a plataforma onde a aplicação será executada.

Cada plataforma possui seu próprio JRE e JVM, formando um multiverso de ambientes. Além disso, o JRE inclui uma API, que é uma biblioteca de componentes com recursos úteis para a execução de aplicações Java.

O JRE deve ser baixado por diferentes grupos de alunos conforme suas plataformas: um grupo para Linux, outro para Windows e um terceiro para macOS. Para um aluno que chega solicitando a aplicação, a orientação é baixar o JRE específico para sua plataforma, como Unix, para acessar a execução da aplicação.

Após a instalação do JRE, o aluno poderá baixar e executar a aplicação. É essencial instalar o JRE na plataforma desejada, pois ele inclui a JVM, que gerencia a execução de aplicações Java nesse ambiente. Assim, a portabilidade do Java é garantida pela JVM.

JVM (JAVA VIRTUAL MACHINE)

A JVM, ou Máquina Virtual Java, é a peça-chave que proporciona a capacidade de multiplataforma para as aplicações Java, permitindo a execução em qualquer lugar após uma única compilação. Ela é responsável por interpretar e executar o bytecode das aplicações compiladas, facilitando a interação com o sistema operacional. Durante a execução, a JVM carrega os arquivos.class, determina a semântica de cada bytecode e executa as operações apropriadas. Os arquivos.class podem funcionar em diversos sistemas operacionais, pois a aplicação precisa ser compilada apenas uma vez para gerar esses arquivos.

Para cada plataforma, existe um JRE que inclui uma JVM específica. Os arquivos.class são únicos e funcionam de maneira igual em todas as plataformas, pois operam sobre a plataforma Java representada pelo JRE. Qualquer organização pode desenvolver uma JVM, desde que siga as especificações para sua construção. Assim, se um novo sistema operacional for criado, a empresa poderá desenvolver sua própria JVM para interagir com ele, respeitando as diretrizes de construção de JVM.

A portabilidade das aplicações Java é acompanhada por uma perda de desempenho em comparação às aplicações compiladas diretamente para uma plataforma específica. Quanto



20m



mais camadas de abstração existirem entre a aplicação e a plataforma, mais lenta será a execução. A comunicação direta entre duas partes é mais eficiente do que a que ocorre por meio de intermediários, que introduzem atrasos no feedback e na resposta.

Analogamente, se uma aplicação Java não opera em uma plataforma específica, pode apresentar uma redução na velocidade e no desempenho em relação a uma aplicação compilada diretamente para essa plataforma. Embora a portabilidade seja uma vantagem, essa característica pode resultar em um desempenho inferior. Portanto, há uma relação inversamente proporcional entre portabilidade e desempenho: ao se optar por maior portabilidade, pode-se sacrificar o desempenho e vice-versa.

JIT (COMPILADOR JUST-IN-TIME)

O JIT (Just-In-Time) é um componente do JRE que melhora o desempenho de aplicativos Java em tempo de execução, compilando bytecodes para código nativo. O desenvolvedor deve continuar a compilar seus arquivos.java uma única vez. O JRE, em conjunto com o JIT, realiza a tarefa de interpretação e execução dos bytecodes.

Alguns itens dos bytecodes são diretamente compilados para código de máquina nativo, mantendo a portabilidade da aplicação. O JIT é ativado quando um método Java é chamado, e os bytecodes desse método são então compilados em código de máquina nativo.

Após a compilação, a JVM executa diretamente o código compilado, em vez de interpretá-lo. Essa melhoria de desempenho ocorre dentro do JRE, enquanto a compilação permanece inalterada, sendo realizada apenas uma vez.

Teoricamente, a compilação de cada método pode aumentar a velocidade do programa Java, aproximando-a da de um aplicativo nativo, ou seja, diretamente compilado para uma plataforma específica. Se a compilação não consumir tempo de processador nem memória, é possível obter um ganho significativo na rapidez das aplicações Java por meio do JIT, que opera dentro do JRE.

No próximo bloco, serão apresentados esquemas visuais e revisões para auxiliar na compreensão dos conceitos abordados, incluindo o JDK, o JRE e a JVM. Esses itens são essenciais, e a revisão permitirá aos alunos entender melhor a codificação Java, além de facilitar a resolução de questões práticas sobre o tema.

Este material foi elaborado pela equipe pedagógica do Gran Concursos, de acordo com a aula preparada e ministrada pelo professor Rogério Gildo Araujo.

A presente gravação tem como objetivo auxiliar no acompanhamento e na revisão do conteúdo ministrado na videoaula. Não recomendamos a substituição do estudo em vídeo pela leitura exclusiva deste material.
